

第一题:

这种题和前面两个序列求第 k 小的方案类似

一般有两种做法, 第一种是二分答案然后验证
这里验证只能用 `dfs` 剪枝验证, 最坏时间复杂度无法证明

第二种是堆模拟, 把当前可能是最大值的值放进堆中, 然后取一个最小值

先排序, 比如最开始是 a_1 , 那么下一个可能的答案只能 a_2 , 不可能是 $a_3 \dots a_n$
下一个是 a_2 , 在下一个只能是 a_3, a_1+a_2 。

那么如果当前是 $a_i+a_j+\dots+a_l$, 那么按照这样子规律下一个可能是 $a_i+a_j+\dots+a_{l+1}$
因为前面小的的下一个已经被拓展了

但是这样有遗漏, 我们这次还要多拓展一点, 在求第 k 大的值的时候顺便可以把第 $k+2$ 大的值也放进去, 比如当前是 a_1 , 我们可以把 a_1+a_2 也放进去, 这样就没有遗漏了。

所以我们每次不仅仅要知道当前和, 还要知道选择的一个最大的数, 然后拓展两种情况
第一次把最大值变为更大的一个, 第二次在最大值的后面再增加一个更大的数。

第二题:

1: 先预处理出来每一行中前 j 个字符中 0 或者 1 的个数

2: 在开一个三位 $f[i][j][k]$ 表示对于第 i 行, 前 j 个格子涂了 k 次颜色的最大正确格子数(只是对于独立的每一行), 再枚举一个上一次涂抹的最后一个格子的位置就可以过了

3: 然后就是一个裸的分组背包(或者泛华背包), 每一行看做一组, 预处理出来的每一行前 m 个格子涂抹 p 次最多正确的格子数就看做一个物品跑一遍就可以确定最大值了

第三题:

如果只看最后两个操作就是一个裸的 `dfs` 序+线段树

如果涉及加点和子树删除呢?

`LCT`? 并不需要

离线线段树就可以了, 和以前讲个的线段树的有效儿子和无效儿子做法一样。